

Capitolo 1 - MongoDB: contesto, installazione e database di esempio

1.1 Che cos'è MongoDB

MongoDB è un database orientato ai documenti. In pratica, invece di organizzare i dati in tabelle fatte di righe e colonne, li organizza in collection che contengono documenti. Un documento è una struttura simile a un oggetto JSON, anche se internamente MongoDB usa un formato binario chiamato BSON. Questo approccio rende molto naturale memorizzare dati ricchi, gerarchici e variabili, come ordini con righe prodotto, clienti con più indirizzi, oppure articoli con attributi diversi a seconda della categoria.

Detto in modo semplice: se in un database relazionale un ordine spesso viene spezzato in più tabelle, in MongoDB una parte importante di quell'informazione può vivere nello stesso documento. Non sempre, ma molto spesso sì. Ed è proprio questa una delle ragioni per cui MongoDB è diventato molto popolare nello sviluppo moderno.

MongoDB mette a disposizione tutte le operazioni che ci aspettiamo da un database serio: inserimento, interrogazione, aggiornamento, cancellazione, indicizzazione, validazione dei dati e aggregazioni avanzate. In più offre un framework di aggregazione molto potente, basato su pipeline di trasformazione dei documenti, che vedremo in profondità nel corso del libro.

Per capirci subito con un esempio, immagina di dover memorizzare un cliente con due indirizzi e tre contatti interni. In un sistema tradizionale potresti avere una tabella clienti, una tabella indirizzi e una tabella contatti. In MongoDB, quello stesso cliente può essere rappresentato come un unico documento che contiene, al suo interno, anche gli indirizzi e i contatti. Questa non è una regola obbligatoria, ma è una possibilità molto tipica del modello documentale.

1.2 MongoDB nel panorama SQL / NoSQL

Quando si parla di database, spesso si parte dalla distinzione tra SQL e NoSQL. È una distinzione utile, ma va capita bene.

I database SQL classici, come PostgreSQL, MySQL o SQL Server, lavorano su un modello relazionale: tabelle, relazioni, join, schema rigido. Sono ottimi quando la struttura dei dati è molto stabile, fortemente normalizzata e quando la coerenza relazionale è il centro del progetto.

MongoDB rientra invece nella famiglia NoSQL, più precisamente nella categoria dei document database. “NoSQL” non significa “senza logica” o “senza struttura”; significa piuttosto che il modello non è quello relazionale classico basato su tabelle. MongoDB usa collection e documenti, con uno schema flessibile, eventualmente controllato da regole di validazione.

Qui è importante evitare un equivoco comune: MongoDB non è un database “senza schema” nel senso assoluto del termine. È più corretto dire che adotta uno schema flessibile. Se vuoi, puoi lasciare i documenti molto liberi; se vuoi più controllo, puoi definire regole di validazione, per esempio usando `$jsonSchema`.

Un esempio concreto chiarisce bene la differenza.

Supponiamo di gestire prodotti informatici. Alcuni prodotti hanno colore e peso, altri hanno capacità di archiviazione, altri ancora hanno diagonale dello schermo. In un database relazionale puoi certamente modellare tutto, ma spesso sei costretto a fare scelte più rigide: colonne opzionali, tabelle secondarie, join aggiuntive. In MongoDB, invece, puoi modellare i prodotti in documenti che condividono un nucleo comune e hanno poi attributi specifici per categoria. È uno dei casi in cui il modello documentale risulta molto naturale.

Questo non significa che MongoDB sia sempre la scelta migliore. Se il dominio richiede una modellazione relazionale molto rigorosa, con vincoli forti e join strutturali continui, un database SQL può restare la soluzione più adatta. Se invece hai dati gerarchici, strutture annidate, campi facoltativi, evoluzione frequente del modello o necessità di lavorare bene con documenti completi, MongoDB è spesso molto efficace. Questa guida non parte dall'idea che MongoDB sia "migliore" in assoluto; parte dall'idea più sana: ogni database ha il suo contesto ideale.

1.3 I concetti base da tenere in mente

Prima ancora di installare tutto, conviene fissare quattro parole chiave:

- **Server:** è il motore database vero e proprio, cioè il processo che memorizza e gestisce i dati.
- **Database:** è un contenitore logico di collections.
- **Collection:** è l'equivalente più vicino a una tabella, anche se il paragone non è perfetto.
- **Documento:** è l'equivalente più vicino a una riga, ma può contenere strutture annidate e array.

Se arrivi dal mondo SQL, un buon modo per orientarti è questo:

- tabella -> collection
- riga -> documento
- colonna -> campo
- join/analisi -> spesso aggregazione, a volte \$lookup

Non è una traduzione perfetta, ma come mappa iniziale funziona bene.

Un'altra parola fondamentale è mongosh. mongosh è la shell moderna di MongoDB: un ambiente interattivo basato su JavaScript e Node.js che ti permette di collegarti a un server MongoDB, lanciare query, esplorare i dati, fare CRUD e testare pipeline di aggregazione. In questo libro la useremo spesso, perché è un modo molto diretto e didattico per vedere cosa fa una query.

1.4 Installare MongoDB sui sistemi operativi più usati

MongoDB fornisce documentazione ufficiale per installare il server su Windows, macOS e Linux, oltre alla documentazione dedicata per mongosh. Qui il nostro obiettivo non è trasformare questo capitolo in un manuale amministrativo completo, ma darti un percorso semplice per arrivare a un ambiente funzionante.

Su Windows, l'approccio più lineare è usare l'installer ufficiale di MongoDB Community Edition e seguire la procedura guidata. In termini pratici, il flusso è questo:

- scarichi MongoDB Community dal canale ufficiale;

- esegui l'installer;
- installi anche mongosh, se non è già incluso nel tuo flusso di setup;
- avvii il servizio MongoDB;
- verifichi di riuscire a collegarti.

L'idea da ricordare è semplice: prima ti serve il server, poi ti serve la shell per lavorarci.

Su macOS la strada più comoda, soprattutto per sviluppatori, è l'installazione tramite Homebrew. In pratica, su Mac il flusso mentale è:

- installi MongoDB con Homebrew;
- verifichi che il server possa partire;
- installi o aggiorni mongosh;
- testi la connessione locale.

Su Linux esistono percorsi diversi a seconda della distribuzione: Ubuntu, Debian, Red Hat, SUSE e altre. Qui il consiglio pratico è molto semplice: non improvvisare con guide vecchie trovate a caso online. Segui sempre il tutorial ufficiale relativo alla tua distribuzione e alla versione corrente, perché repository, nomi dei pacchetti e passaggi possono cambiare nel tempo.

1.5 Installare mongosh

mongosh merita un paragrafo tutto suo, perché nel libro lo useremo continuamente.

MongoDB Shell è un ambiente REPL basato su JavaScript e Node.js, progettato per connettersi a deployment locali, remoti o su Atlas. Serve per testare query, fare CRUD, eseguire aggregazioni, scrivere script e ispezionare rapidamente il contenuto del database.

Concettualmente, pensa a mongosh come al banco di lavoro del libro. Il server è il motore. mongosh è il posto in cui metti le mani sui dati.

1.6 Verificare che MongoDB sia installato correttamente

Una volta installati server e shell, il primo obiettivo non è fare query complicate. Il primo obiettivo è capire se tutto è in piedi.

La verifica minima è questa:

- il server MongoDB è avviato;
- mongosh parte;
- riesci a connetterti al server locale.

Se il collegamento funziona, hai già superato la parte più noiosa del setup. Da quel momento puoi iniziare a lavorare davvero.

Un errore tipico del principiante è confondere questi due problemi:

- mongosh non parte
- mongosh parte, ma non riesce a collegarsi al server

Sembrano simili, ma sono due problemi diversi. Nel primo caso manca o non funziona la shell. Nel secondo caso la shell c'è, ma il processo MongoDB non è avviato oppure non è raggiungibile.

1.7 Introduzione al database di esempio

Per tutto il libro useremo un database di esempio che rappresenta una piccola-media azienda commerciale. È una scelta intenzionale: un dominio aziendale è abbastanza realistico da produrre query interessanti, ma abbastanza intuitivo da essere capito anche da chi non conosce ancora MongoDB.

Il database descrive, a grandi linee, queste aree:

- clienti
- fornitori
- prodotti
- ordini cliente
- ordini di acquisto
- magazzini
- movimenti di magazzino
- dipendenti

Il punto importante non è memorizzare tutto il mondo, ma avere un dominio che ci permetta di fare esempi credibili.

Per esempio:

- vuoi trovare i clienti attivi del Nord Italia;
- vuoi elencare i prodotti sotto la soglia di riordino;
- vuoi vedere quali fornitori consegnano con lead time più lunghi;
- vuoi calcolare il fatturato mensile;
- vuoi capire in quale magazzino si muove di più una certa categoria di prodotti.

Con un dataset del genere possiamo coprire non solo le query base, ma anche i casi più interessanti di aggregazione.

1.8 Come è fatto il database, spiegato senza tecnicismi

Immagina il database come una piccola mappa dell'azienda.

C'è una collezione che raccoglie i clienti. Ogni cliente ha una propria identità, recapiti, stato commerciale, eventuali indirizzi e contatti.

C'è una collezione dei fornitori. Qui troviamo chi vende merce all'azienda, con informazioni come provenienza, tempi medi di consegna e categorie di prodotti trattate.

Poi abbiamo i prodotti, cioè il catalogo di ciò che l'azienda compra e rivende. Ogni prodotto appartiene a una categoria, ha un prezzo di vendita, un costo, un fornitore di riferimento e alcune caratteristiche descrittive.

Ci sono i magazzini, perché i prodotti non stanno nel vuoto: devono essere conservati da qualche parte. A quel punto entrano in scena anche i movimenti di magazzino, cioè gli ingressi, le uscite, i trasferimenti e le rettifiche.

Infine ci sono gli ordini. Alcuni sono ordini ricevuti dai clienti, altri sono ordini emessi verso i fornitori. In entrambi i casi, rappresentano il cuore operativo dell'azienda.

Volendo usare una metafora, puoi vedere il database come una città:

- i clienti e i fornitori sono le persone con cui l'azienda ha rapporti;
- i prodotti sono gli oggetti che circolano;
- gli ordini sono gli eventi economici;
- i magazzini sono i luoghi fisici;
- i movimenti di magazzino sono gli spostamenti;
- i dipendenti sono gli attori interni che gestiscono il sistema.

Questa immagine mentale ti aiuterà molto quando inizieremo con le query vere.

1.9 Perché questo database è utile per imparare

Un dataset didattico deve essere realistico, ma non caotico.

Se fosse troppo semplice, per esempio con una sola collection di prova e tre campi in croce, non potremmo mostrare join logici, aggregazioni, array, strutture annidate e casi reali.

Se fosse troppo complicato, il lettore si perderebbe prima ancora di imparare le query.

Il database scelto sta in mezzo: è abbastanza ricco da permettere esempi seri, ma abbastanza chiaro da restare leggibile.

Per esempio, nei primi capitoli potremo fare cose molto semplici, come cercare tutti i clienti attivi oppure tutti i prodotti di una certa categoria. Più avanti potremo passare a domande come: “quali sono i tre clienti che hanno generato più fatturato nell’ultimo trimestre?” oppure “qual è il valore complessivo della merce movimentata per magazzino?”. Queste domande richiedono aggregazioni, raggruppamenti e talvolta più collections. Ed è proprio per questo che il dataset è stato costruito in quel modo.

1.10 Installare il database di esempio

Nel materiale operativo del libro useremo uno script di setup che crea il database e lo popola con un numero consistente di documenti.

Qui non serve ancora capire il contenuto tecnico dello script riga per riga. In questa fase ti basta sapere che lo script:

- crea il database di esempio;
- crea le collection principali;
- inserisce un insieme realistico di dati;
- prepara il terreno per tutti gli esercizi successivi.

Dal punto di vista pratico, il flusso è semplice:

- apri mongosh;
- lanci lo script di setup;
- aspetti che il caricamento termini;

- controlli che le collection esistano;
- controlli che i conteggi tornino.

L'idea è la stessa di quando installi un software e poi lanci un test iniziale: non ti interessa ancora conoscere l'implementazione interna, ti interessa verificare che l'ambiente sia pronto.

1.11 Verificare che il caricamento sia andato a buon fine

Questa è una fase molto importante. Molti saltano il controllo iniziale e poi, più avanti, quando una query non restituisce il risultato atteso, non sanno se il problema è nella query o nei dati caricati.

Dopo l'installazione del database, la prima cosa da fare è verificare che le collection principali esistano davvero.

Poi conviene fare un secondo controllo: contare i documenti presenti in ciascuna collection.

Per esempio, in mongosh puoi eseguire query di questo tipo:

```
use company_demo
```

```
db.clients.countDocuments()  
db.suppliers.countDocuments()  
db.products.countDocuments()  
db.salesOrders.countDocuments()  
db.purchaseOrders.countDocuments()  
db.warehouses.countDocuments()  
db.inventoryMovements.countDocuments()  
db.employees.countDocuments()
```

Queste query sono volutamente semplici. Non servono per fare analisi, ma per verificare che il caricamento sia corretto.

Se, per esempio, ti aspetti alcune centinaia di clienti e prodotti, ma ottieni 0, hai già capito che qualcosa non è andato bene nel setup. Se invece i numeri sono plausibili e coerenti con il dataset atteso, puoi procedere con molta più tranquillità.

Un altro controllo utile è elencare i nomi delle collection presenti:

```
show collections
```

E un test ancora più concreto è visualizzare alcuni documenti di esempio, senza ancora preoccuparti dei dettagli:

```
db.clients.find().limit(3)  
db.products.find().limit(3)  
db.salesOrders.find().limit(2)
```

Questo passaggio ha un grande valore psicologico oltre che tecnico: ti fa vedere che il database non è più un concetto astratto. C'è davvero, contiene dati reali e puoi già iniziare a interrogarlo.

1.12 Cosa fare se qualcosa non torna

Se una delle query di controllo non restituisce il risultato atteso, niente panico. Nella maggior parte dei casi il problema è uno di questi:

- non sei connesso al server giusto;
- il server MongoDB non è avviato;
- non hai eseguito correttamente lo script di setup;
- stai usando un database diverso da `company_demo`;
- il caricamento si è interrotto a metà.

Il metodo migliore è controllare una cosa per volta.

Prima verifica la connessione. Poi verifica il database corrente. Poi verifica la presenza delle collection. Solo dopo passa al controllo dei numeri.

È lo stesso approccio che useresti nel debugging di un programma: partire dalle condizioni di base e salire gradualmente.

1.13 Conclusione del capitolo

A questo punto hai costruito le fondamenta del libro.

Hai visto che MongoDB è un database documentale basato su collection e documenti, che si colloca nel mondo NoSQL ma non è un sistema senza struttura. Hai visto che può usare uno schema flessibile, eventualmente rafforzato con regole di validazione, e che mette a disposizione sia CRUD sia un framework di aggregazione molto potente.

Hai anche chiarito il quadro operativo: serve un server MongoDB, serve mongosh, e serve un database di esempio da caricare e verificare.

Infine hai conosciuto il dominio aziendale che farà da filo conduttore per tutto il libro. Da qui in avanti non lavoreremo su esempi scolastici astratti, ma su un dataset pensato per simulare casi reali: clienti, fornitori, prodotti, ordini, magazzini e movimenti.

Nel prossimo capitolo entreremo davvero nel cuore del libro: inizieremo a parlare di query MongoDB in senso operativo, distinguendo tra operazioni CRUD, linguaggio di interrogazione e aggregation framework.